Ultimate Java DSA Master Study Guide

1. Data Types & Ranges

Understanding Java primitive data types and their ranges (approximate maximum values shown in powers of 10):

Data Type	Size (bits)	Range	Approximate Max Value (≈ 10^x)
byte	8	-128 to 127	~ 102
short	16	-32,768 to 32,767	~ 104
int	32	-2,147,483,648 to 2,147,483,647	~ 109
long	64	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	~ 1018
float	32	±1.4e-45 to ±3.4e38 (approx.)	~ 1038
double	64	±4.9e-324 to ±1.7e308 (approx.)	~ 10308
char	16	0 to 65,535 (Unicode characters)	~ 104
boolean	1 (conceptual)	true or false	N/A

2. Input/Output (Scanner)

The Scanner class reads input from various sources, commonly System.in.

- nextInt(): Reads next token as an integer.
- nextLong(): Reads next token as a long.
- nextDouble(): Reads next token as a double.
- next(): Reads next token as a string (up to whitespace).
- nextLine(): Reads entire line as a string including spaces.
- hasNext(): Checks if more tokens are available.

3. Time Complexity vs. Input Size (Big O Limits)

Which algorithm complexities are practical given input size n:

Input Size n	Acceptable Time Complexity
<= 10	Any complexity (even factorial $O(n!)$)
<= 20	Exponential algorithms $O(2^n)$, backtracking possible
<= 100	Polynomial up to $O(n^3)$ feasible
<= 500	Approximately $O(n^2)$ algorithms possible
<= 10^4	Linearithmic $O(n \log n)$ or better algorithms
<= 10^6	Linear $O(n)$ or logarithmic $O(\log n)$ algorithms
> 10^6	Constant $O(1)$ or near-constant time, heavy optimization needed

4. Strings (Immutable)

Strings in Java are immutable, meaning any modification creates a new object. Common methods and their time complexities:

- charAt(int index): **O(1)** – direct index access.
- substring(int beginIndex) / substring(int beginIndex, int endIndex): **O(n)** – creates new String with copied characters.
- equals(Object obj): **O(n)** – compares characters sequentially.
- split(String regex): **O(n)** – splits based on pattern, traverses the string.
- indexOf(String str): **O(n)** – search for substring.
- toLowerCase(), toUpperCase(), trim(): **O(n)** – process all chars.

5. StringBuilder (Mutable)

StringBuilder allows mutable string operations with less overhead than String concatenations:

- append(X x): **O(1)** amortized – appends at end.
- insert(int offset, X x): **O(n)** – inserts shifts characters.
- delete(int start, int end): **O(n)** – removes substring.
- deleteCharAt(int index): **O(n)** – removes single char and shifts.
- replace(int start, int end, String str): **O(n)** – replaces substring.
- reverse(): **O(n)** – reverses character sequence.
- toString(): **O(n)** – converts builder to immutable String.

6. Arrays (java.util.Arrays)

Utility methods for arrays and their time complexities:

- Arrays.sort(array): **O(n \log n)** – Dual-Pivot Quicksort for primitives.
- Arrays.binarySearch(array, key): **O(\log n)** – Requires sorted array.
- Arrays.fill(array, val): **O(n)** – Sets all elements.
- Arrays.equals(array1, array2): **O(n)** – Element-wise comparison.
- Arrays.copyOf(original, newLength): **O(n)** – Copies and resizes.

7. ArrayList, HashSet, HashMap

ArrayList (Resizable array implementation)

- add(E e): Amortized **O(1)**.
- remove(int index): **O(n)** – shifts elements left.
- get(int index): **O(1)**.
- set(int index, E element): **O(1)**.
- contains(Object o): **O(n)** – linear search.

HashSet (Hash-backed, no duplicates)

- add(E e): Average **O(1)**.
- contains(Object o): Average **O(1)**.
- remove(Object o): Average **O(1)**.

HashMap (Key-Value Hash Map)

- put(K key, V value): Average **O(1)**.
- get(Object key): Average **O(1)**.
- remove(Object key): Average **O(1)**.
- computeIfAbsent(K key, Function): Average **O(1)** plus computation.
- putIfAbsent(K key, V value): Average **O(1)**.

8. Ordered Collections (TreeSet, TreeMap)

Both are based on Red-Black Trees, supporting naturally ordered keys and efficient navigation with **O(\log n)** time for key operations.

- ceiling(key): Smallest element ≥ key.
- floor(key): Largest element ≤ key.
- higher(key): Smallest element > key.
- lower(key): Largest element < key.
- Other operations (add, remove, contains) also **O(\log n)**.

9. Queue, Deque, PriorityQueue

Queue & PriorityQueue

- offer(E e) / add(E e): Add element at tail, **O(1)** for Queue, **O(\log n)** for PriorityQueue.
- poll() / remove(): Remove and return head, **O(1)** Queue, **O(\log n)** PriorityQueue.
- peek() / element(): Return head without removing, **O(1)**.

Deque (Double-ended Queue, e.g., LinkedList, ArrayDeque)

- addFirst(E e) / offerFirst(E e): Insert at front, **O(1)**.
- addLast(E e) / offerLast(E e): Insert at end, **O(1)**.
- removeFirst() / pollFirst(): Remove from front, **O(1)**.
- removeLast() / pollLast(): Remove from end, **O(1)**.
- getFirst() / peekFirst(): Peek front element, **O(1)**.
- getLast() / peekLast(): Peek last element, **O(1)**.

10. Custom Sorting (Comparators)

Java supports custom sorting using Comparator interfaces, often with lambda expressions:

```
// Single-level comparator for sorting by age ascending
Comparator byAge = (p1, p2) -> Integer.compare(p1.getAge(), p2.getAge());

// Multi-level comparator: by age ascending, then name lex ascending
Comparator multiLevel = Comparator
    .comparingInt(Person::getAge)
    .thenComparing(Person::getName);
```

Usage example with Collections.sort(list, comparator) or list.sort(comparator).

11. Math, Bit Manipulation, and Conversions

Math Utilities

- Math.min/max(a, b): returns minimum/maximum.
- Math.abs(x): absolute value.
- Math.pow(base, exp): power function.
- Math.sqrt(x): square root.
- Math.log(x), Math.log10(x): logarithms.

Bit Manipulation

- Bitwise operators:
  - & (AND), | (OR), ^ (XOR)
  - ~ (NOT)
  - << (left shift), >> (signed right shift), >>> (unsigned right shift)
- Integer.bitCount(int n): counts number of set bits, runs in **O(1)**.

Conversions

- **String <-> Integer:**
  - Integer.parseInt(String), Integer.toString(int)
- **Char <-> Integer:**
  - Explicit cast: (int) char to get ASCII/Unicode code.
  - Convert digit character to int: char - '0'.
- **List <-> Array:**
  - list.toArray(new Type[0]) converts List to array.
  - Arrays.asList(array) converts array to List.

```
Scanner sc = new Scanner(System.in);
String word = sc.next(); // reads a word
String line = sc.nextLine(); // reads a line (note: may require extra sc.nextLine() to
consume newline)
int n = sc.nextInt();
long bigNum = sc.nextLong();
double val = sc.nextDouble();
char c = sc.next().charAt(0);
```

3. Strings (Immutable)

Strings in Java are immutable and provide various utility methods:

Initialization	String s = "hello"; or new String("hello");
Basic Operations	length(), charAt(int), substring(beginIndex), substring(beginIndex, endIndex), isEmpty()
Comparison	equals(Object), equalsIgnoreCase(String), compareTo(String), startsWith(String), endsWith(String), contains(CharSequence)
Modification	toLowerCase(), toUpperCase(), trim(), replace(char, char), replace(CharSequence, CharSequence), replaceAll(String regex, String replacement)
Searching	indexOf(String), indexOf(String, int), lastIndexOf(String), lastIndexOf(String, int), matches(String regex)
Splitting & Joining	split(String regex), split(String regex, int limit), join(CharSequence delimiter, CharSequence... elements)
Conversion	toCharArray() converts string into char array

Conversion	toCharArray() converts string into char array

Example:

```
String s = "JavaDSA";
char c = s.charAt(2); // 'v'
String sub = s.substring(4); // "DSA"
boolean starts = s.startsWith("Java"); // true
String replaced = s.replace('a', '@'); // "J@v@DS@"
String[] parts = s.split("A"); // ["JavaDS", ""]
```

4. StringBuilder (Mutable)

StringBuilder is used for mutable sequence of characters, efficient for modifications like append, insert, delete, and backtracking.

- **Initialization:**
  - StringBuilder sb = new StringBuilder(); (capacity 16)
  - StringBuilder sb = new StringBuilder("initial");
- **Key methods:**
  - append(X x): add at end
  - insert(int offset, X x): insert at offset
  - delete(int start, int end): remove substring
  - deleteCharAt(int index): remove character
  - replace(int start, int end, String str): replace substring
  - setCharAt(int index, char ch): change character
  - reverse(): reverse sequence
  - setLength(int newLength): truncate or extend (key for backtracking)
  - toString(): convert to String

Backtracking Example (using setLength**):**

5. Arrays (java.util.Arrays)

- **Sorting:**
  - Arrays.sort(array): sorts entire array ascending
  - Arrays.sort(array, fromIndex, toIndex): sorts subrange [fromIndex, toIndex)
  - Arrays.parallelSort(array): sorts using multiple threads
- **Searching:**
  - Arrays.binarySearch(array, key)
  - Arrays.binarySearch(array, fromIndex, toIndex, key)
- **Filling:**
  - Arrays.fill(array, val): fill entire array
  - Arrays.fill(array, fromIndex, toIndex, val)
- **Equality and Hashing:**
  - Arrays.equals(array1, array2): compare arrays
  - Arrays.hashCode(array)
- **Copying:**
  - Arrays.copyOf(original, newLength)
  - Arrays.copyOfRange(original, from, to)
- **Conversion:**
  - Arrays.asList(T... a): converts array to List
- **Streams:**
  - Arrays.stream(array): sequential stream
  - Arrays.parallelSort(array) (concurrent)

6. Collections Framework

ArrayList (and List Interface)

- **Initialization:**
  - ArrayList<Integer> list = new ArrayList<>();
- **Basic Methods:**
  - size(), isEmpty()
  - get(int index), set(int index, E element)
  - add(E e), add(int index, E element)
  - remove(int index), remove(Object o)
  - clear()
- **Searching:**
  - contains(Object o), indexOf(Object o), lastIndexOf(Object o)
- **Bulk Operations:**
  - addAll(Collection c), addAll(int index, Collection c)
  - removeAll(Collection c), retainAll(Collection c)
  - containsAll(Collection c)
- **List Views:**
  - subList(int fromIndex, int toIndex) returns a view of portion of list
- **Streams & Iteration:**
  - stream()
  - forEach(Consumer action)
  - filter(Predicate) removes elements matching predicate
  - sort(Comparator)

HashSet (and Set Interface)

- **Initialization:**
  - HashSet<Integer> set = new HashSet<>();
- **Basic Methods:**
  - size(), isEmpty(), add(E e), remove(Object o), clear(), contains(Object o)
  - iterator(): traverse elements
- **Bulk Operations:**
  - addAll(Collection c), removeAll(Collection c), retainAll(Collection c), containsAll(Collection c)
- **Streams:** stream() for sequential iteration

HashMap (and Map Interface)

- **Initialization:**
  - HashMap<K, V> map = new HashMap<>();
- **Basic Methods:**
  - size(), isEmpty(), put(K key, V value), get(Object key), remove(Object key), clear()
  - getOrDefault(Object key, V defaultValue)
- **Content Checks:**
  - containsKey(Object key), containsValue(Object value)
- **Bulk Operations:**
  - putAll(Map m)
  - putIfAbsent(K key, V value): adds mapping if key absent
  - remove(Object key, Object value): removes key only if currently mapped to value
- **Views:**
  - keySet(): returns Set of keys
  - values(): returns Collection of values
  - entrySet(): returns Set of Map.Entry<K,V>

- **Views:**
  - keySet(): returns Set of keys
  - values(): returns Collection of values
  - entrySet(): returns Set of Map.Entry<K,V>
- **Streams & Iteration:**
  - forEach(BiConsumer) to process entries
  - replaceAll(BiFunction) to update values

7. Advanced Map Operations

These methods help cleanly manipulate map entries:

- computeIfAbsent(K key, Function mappingFunction)
- If the key is not present, computes value and inserts it.

```
map.computeIfAbsent("apple", k -> new ArrayList<>()).add("fruit");
```

- putIfAbsent(K key, V value)
- Put key-value pair only if key is absent.

```
map.putIfAbsent("banana", 10);
```

- merge(K key, V value, BiFunction remappingFunction)
- Merge value with existing value if present, else insert value.

```
map.merge("count", 1, Integer::sum);
```

8. Ordered Collections (TreeSet & TreeMap)

TreeSet and TreeMap maintain natural ascending order or ordering defined by a comparator.

- **Navigation Methods:**
  - ceiling(E e): least element ≥ e, or null
  - floor(E e): greatest element ≤ e, or null
  - higher(E e): least element > e
  - lower(E e): greatest element < e
  - pollFirst(), pollLast(): remove & return first or last element
  - first(), last(): retrieve first or last element
- **Initialization with Comparator:**

```
TreeSet<Integer> ts = new TreeSet<>((a, b) -> b - a); // descending order
```

9. Queue, Deque & PriorityQueue

Queue Interface & PriorityQueue:

- add(E e) and offer(E e): add element to queue
  - add(E e) throws exception if full
  - offer(E e) returns false if full (safer)
- remove() and poll(): remove and return head
  - remove() throws exception if empty
  - poll() returns null if empty
- element() and peek(): retrieve head without removing
  - element() throws exception if empty
  - peek() returns null if empty
- size(), isEmpty()

PriorityQueue by default implements a min-heap.

9. Queue, Deque & PriorityQueue

Queue Interface & PriorityQueue:

- add(E e) and offer(E e): add element to queue
  - add(E e) throws exception if full
  - offer(E e) returns false if full (safer)
- remove() and poll(): remove and return head
  - remove() throws exception if empty
  - poll() returns null if empty
- element() and peek(): retrieve head without removing
  - element() throws exception if empty
  - peek() returns null if empty
- size(), isEmpty()

PriorityQueue by default implements a min-heap.

```
// Min heap (natural order)
PriorityQueue<Integer> minHeap = new PriorityQueue<>();

// Max heap (reverse comparator)
PriorityQueue<Integer> maxHeap = new PriorityQueue<>((a, b) -> b - a);
```

Deque Interface (implemented by LinkedList or ArrayDeque):

- addFirst(E e), offerFirst(E e): insert at front
- addLast(E e), offerLast(E e): insert at end
- removeFirst(), pollFirst(): remove first element
- removeLast(), pollLast(): remove last element
- getFirst(), peekFirst(): retrieve first element
- getLast(), peekLast(): retrieve last element

10. Custom Sorting (Comparators)

Java allows sorting with custom comparator logic, including multi-level sorting.

- **Lambda Example:** Sort by value ascending:

```
list.sort((a, b) -> a.val - b.val);
```

- **Multi-level sorting:** sort by val, then by id if val equal:

```
list.sort((a, b) -> {
    int cmp = Integer.compare(a.val, b.val);
    if (cmp != 0) return cmp;
    return Integer.compare(a.id, b.id);
});
```

- **Comparator chaining:**

```
Comparator<Item> cmp = Comparator.comparingInt(Item::getVal)
        .thenComparingInt(Item::getId);
Collections.sort(list, cmp);
```

11. Math & Bit Manipulation

- **Math Class Methods:**
  - Math.abs(x): absolute value
  - Math.max(a, b), Math.min(a, b)
  - Math.pow(a, b): a raised to power b
  - Math.sqrt(x)
  - Math.log(x), Math.log10(x)
  - Math.ceil(x), Math.floor(x), Math.round(x)
  - Math.random(): random double between 0.0 and 1.0
  - Math.max and Math.min for comparisons
- **Bitwise Operators:**
  - &: bitwise AND
  - |: bitwise OR
  - ^: bitwise XOR
  - ~: bitwise NOT
  - <<: left shift (multiply by powers of 2)
  - >>: signed right shift (divide by powers of 2)
  - >>>: unsigned right shift

  Use cases:
  - Check if odd/even: (x & 1) == 1 is odd
  - Power of two check: (x & (x – 1)) == 0
  - Set bit: x | (1 << pos)
  - Clear bit: x & ~(1 << pos)
  - Toggle bit: x ^ (1 << pos)

12. Conversions

Conversion	Method	Example
String to int	Integer.parseInt(String)	int n = Integer.parseInt("123");
String to long	Long.parseLong(String)	long l = Long.parseLong("123456");
String to double	Double.parseDouble(String)	double d = Double.parseDouble("3.14");
int to String	String.valueOf(int) or Integer.toString(int)	String s = String.valueOf(123);
char to String	Character.toString(char) or "" + char	String ch = Character.toString('a');
List to Array	list.toArray(new T[0])	Integer[] arr = list.toArray(new Integer[0]);
Array to List	Arrays.asList(array)	List<String> list = Arrays.asList(strArr);
String to char array	str.toCharArray()	char[] c = s.toCharArray();
char array to String	new String(charArray)	String s = new String(cArr);
Primitive array to List (boxed)	Use Java 8 Streams	List<Integer> list = Arrays.stream(arr).boxed().collect(Collectors.toList());

5. Algorithm Selection Based on Input Size and Complexity

- **Up to ~10**: Algorithms with O(n!) or O(2^n) can be used (e.g., permutations).
- **Up to ~103**: Algorithms with O(n^3) or O(2^n) with pruning might be feasible.
- **Up to ~105**: Algorithms with O(n \log n), O(n \sqrt n) possible.
- **Up to ~107**: Linear O(n) or near-linear algorithms in efficient languages.
- **Sorting**: O(n \log n) sorts capable up to 107 depends on language and time.
- **Constant or logarithmic complexity: Ideal for very large inputs (e.g., O(1), O(\log n)).**

3. Java Methods for Data Structures and Strings

3.1 String Methods

- length(): Returns length of the string.
- charAt(int index): Returns char at given index.
- substring(int beginIndex), substring(int beginIndex, int endIndex): Returns substring.
- isEmpty(): Checks if length is 0.
- toCharArray(): Converts to char array.
- equals(Object obj), equalsIgnoreCase(String str): String comparison.
- compareTo(String str): Lexicographical comparison, returns int.
- startsWith(String prefix), endsWith(String suffix): Checks string prefixes/suffixes.
- contains(CharSequence seq): Checks substring presence.
- toLowerCase(), toUpperCase(): Case conversion.
- trim(): Removes leading/trailing whitespace.
- replace(char oldChar, char newChar), replace(CharSequence target, CharSequence replacement), replaceAll(String regex, String replacement): Replace characters or patterns.
- indexOf(String str), indexOf(String str, int fromIndex), lastIndexOf(String str), lastIndexOf(String str, int fromIndex): Searching.
- matches(String regex): Regex pattern matching.
- split(String regex), split(String regex, int limit): Splitting strings to arrays.
- join(CharSequence delimiter, CharSequence... elements) (static): Joins elements with delimiter.

3.2 StringBuilder Methods

- Constructors: StringBuilder(), StringBuilder(String str).
- length(), capacity(): Length and capacity.
- charAt(int index), substring(int start), substring(int start, int end).
- append(X x): Append any type's string representation.
- insert(int offset, X x): Insert string rep. at offset.
- delete(int start, int end), deleteCharAt(int index): Remove chars.
- replace(int start, int end, String str): Replace substring.
- setCharAt(int index, char ch): Set char at index.
- reverse(): Reverse sequence.
- setLength(int newLength): Set length (truncate/expand).
- toString(): Convert to String.

toString(): Convert to String.

3.3 Arrays (java.util.Arrays)

- Arrays.toString(array): String representation.
- Arrays.equals(array1, array2): Equality check.
- Arrays.hashCode(array): Hash code by contents.
- Arrays.sort(array): Sort in ascending order; overloads for ranges and custom comparators.
- Arrays.parallelSort(array): Multi-threaded sort.
- Arrays.binarySearch(array, key): Binary search on sorted array; overloads for range.
- Arrays.fill(array, val): Fill array or range with value.
- Arrays.asList(T... a): Convert array to List.
- Arrays.copyOf(original, newLength), Arrays.copyOfRange(original, from, to): Copy & resize/partial copy.
- Arrays.stream(array): Stream from array.

3.4 ArrayList (and List) Methods

- Constructors: ArrayList(), ArrayList(Collection c).
- size(), isEmpty().
- get(int index), set(int index, E element).
- add(E e), add(int index, E element).
- remove(int index), remove(Object o).
- clear(): Remove all elements.
- contains(Object o), indexOf(Object o), lastIndexOf(Object o).
- Bulk operations: addAll, removeAll, retainAll, containsAll.
- subList(int fromIndex, int toIndex): View of portion of list.
- Java 8 stream operations: stream(), forEach, filter, sort(Comparator c).

3.5 HashSet (and Set) Methods

- Constructors: HashSet(), HashSet(Collection c).
- size(), isEmpty().
- add(E e), remove(Object o), clear().
- contains(Object o).
- Bulk operations like addAll, removeAll, retainAll, containsAll.
- Iteration: iterator(), Java 8 stream() and forEach.

3.6 HashMap (and Map) Methods

- Constructors: HashMap(), HashMap(Map m).
- size(), isEmpty().
- put(K key, V value), get(Object key), getOrDefault(Object key, V defaultValue).
- remove(Object key), clear().
- Check presence: containsKey(Object key), containsValue(Object value).
- Bulk and conditional updates:
  - putIfAbsent(K key, V value)
  - compute(K key, BiFunction remappingFunction)
  - computeIfAbsent(K key, Function mappingFunction)
  - computeIfPresent(K key, BiFunction remappingFunction)
  - merge(K key, V value, BiFunction remappingFunction)
  - replaceAll(BiFunction)
- Views:
  - keySet()
  - values()
  - entrySet()
- Java 8 iteration: forEach(BiConsumer), streams.

3.7 Queue and PriorityQueue Methods

- add(E e), offer(E e): Add element (offer returns boolean, add throws exception if full).
- remove(), poll(): Remove and return head (poll returns null if empty).
- element(), peek(): Retrieve head without removal.
- size(), isEmpty().
- PriorityQueue is a min-heap by default; custom comparator for different ordering.

3.8 Stack Methods

- push(E item): Push item onto stack.
- pop(): Remove and return top element.
- peek(): Retrieve top without remove.
- empty(): Check if empty.
- search(Object o): Position of an object (1-based from top).

- search(Object o): Position of an object (1-based from top).

3.9 Deque (LinkedList, ArrayDeque) Methods

- Add:
  - addFirst(E e), offerFirst(E e): Insert at front.
  - addLast(E e), offerLast(E e): Insert at end.
- Remove:
  - removeFirst(), pollFirst()
  - removeLast(), pollLast()
- Retrieve:
  - getFirst(), peekFirst()
  - getLast(), peekLast()